

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 8

Task 1 – Explore the code structure (reading)

1. In the OpenGL widget/window class
 - Which method is called once to create the CityEngine?

```
177 # ----- QOpenGLWindow overrides -----
178
179 def initializeGL(self):
180     self._patch_pygame_display()
181
182     dpr = self.devicePixelRatio()
183     w = max(1, int(self.width() * dpr))
184     h = max(1, int(self.height() * dpr))
185     aspect = w / h
186
187     self.engine = CityEngine(
188         aspect_ratio=aspect,
189         detail=self.detail,
190         shadows=self.use_shadows,
191     )
192     self.engine.renderer._window_size = (w, h)
193     self.last_time = time.perf_counter()
```

```
219
220 # update renderer size to physical size (for glViewport)
221 self.engine.renderer._window_size = (phys_w, phys_h)
222
```

- Which method is called every frame to render the scene?

```

223     def paintGL(self):
224         if self.engine is None:
225             return
226
227         if self.view_mode == "sky":
228             self.engine.render_sky()
229         else:
230             self.engine.render_main()
231

```

- Which method is called periodically to update the simulation time?

```

81
82     # timer to drive simulation & repaint
83     self.timer = QTimer(self)
84     self.timer.timeout.connect(self._on_tick)
85     self.timer.start(16) # ~60 FPS
86

```

2. In city_engine.py:

- Where is the main camera rig created and positioned?

```

77
78     # Cameras
79     self.camera = Camera(aspect_ratio=self.aspect_ratio)
80     self.rig = MovementRig(units_per_second=12)
81     self.rig.add(self.camera)
82     self.scene.add(self.rig)
83     self.rig.set_position([0, 12, 45])
84     self.camera.look_at([0, 0, 0])
85

```

- Where is the traffic light state updated?

```

395 # ----- Traffic light state -----
396 cycle = self.time % self.traffic_cycle
397 ns_green = cycle < self.traffic_ns_green_time
398 ew_green = not ns_green
399
400 red_on = [1.0, 0.1, 0.1]
401 red_off = [0.2, 0.0, 0.0]
402 green_on = [0.2, 1.0, 0.2]
403 green_off = [0.0, 0.2, 0.0]
404
405 for m in self.ns_red_bulbs:
406     m.material.set_properties(
407         {"baseColor": red_on if not ns_green else red_off}
408     )
409 for m in self.ns_green_bulbs:
410     m.material.set_properties(
411         {"baseColor": green_on if ns_green else green_off}
412     )
413 for m in self.ew_red_bulbs:
414     m.material.set_properties(
415         {"baseColor": red_on if not ew_green else red_off}
416     )
417 for m in self.ew_green_bulbs:
418     m.material.set_properties(
419         {"baseColor": green_on if ew_green else green_off}
420     )

```

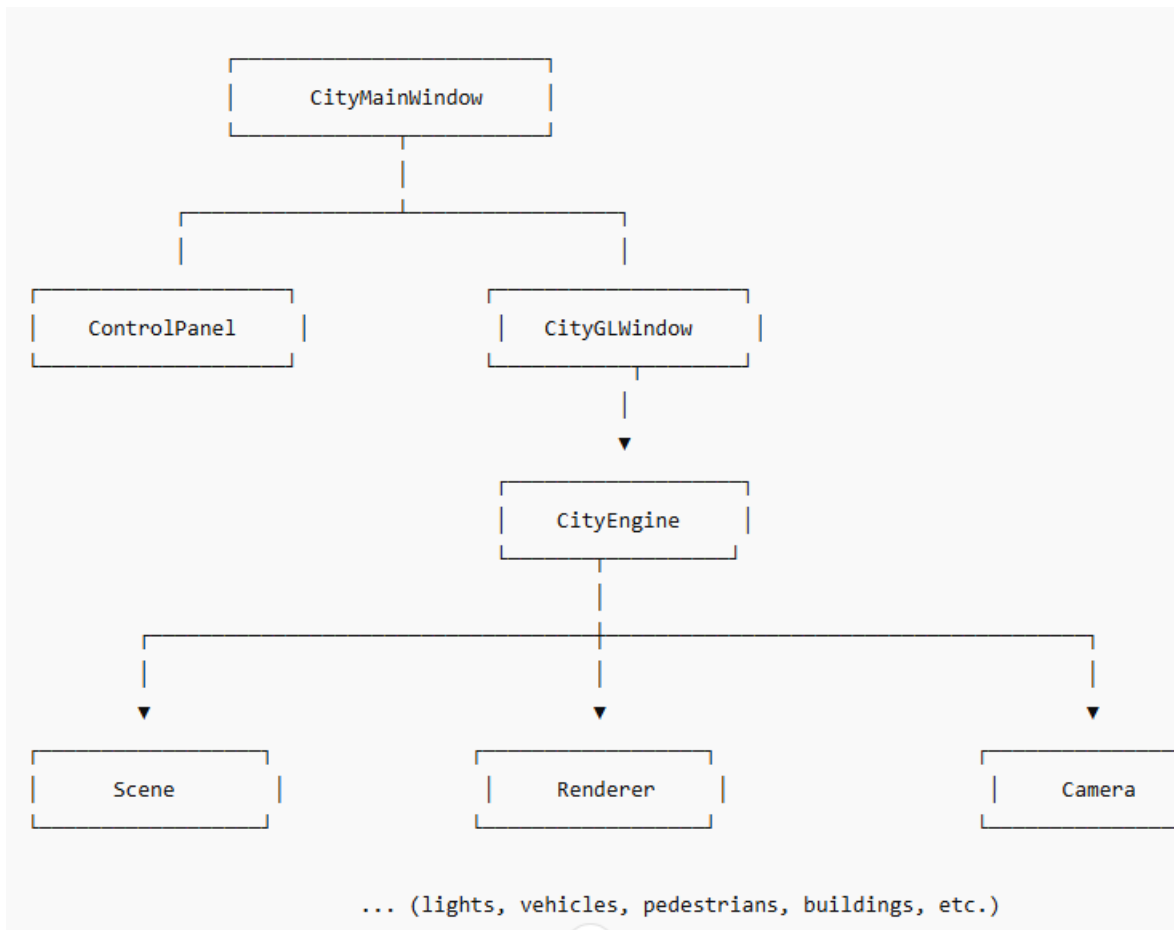
- Where are vehicles and pedestrians initialized?

```

city_engine.py > CityEngine > _build_world
47 class CityEngine:
122     def _build_world(self):
347
348         # along N-S roads
349         for x in self.road_x:
350             for z in range(-80, 81, 10):
351                 for sign in (+1, -1):
352                     tx = x + sign * tree_offset
353                     tz = z
354                     if self.is_near_intersection(tx, tz, clear_margin):
355                         continue
356                     self.make_tree(tx, tz, small=True)
357
358         # ===== Vehicles & Pedestrians =====
359         self.init_vehicles()
360         self.init_pedestrians()
361

```

3. Draw a simple diagram:



Task 2 – Camera preset buttons

2.1. Add widgets

```
290 # --- Camera Presets ---
291 group = QGroupBox("Camera presets")
292 h = QHBoxLayout()
293
294 self.btn_view_front = QPushButton("Front view")
295 self.btn_view_left = QPushButton("Left view")
296 self.btn_view_top = QPushButton("Top view")
297
298 h.addWidget(self.btn_view_front)
299 h.addWidget(self.btn_view_left)
300 h.addWidget(self.btn_view_top)
301
302 group.setLayout(h)
303 layout.addWidget(group)
304
305 self.btn_view_front.clicked.connect(self.gl_window.set_view_front)
306 self.btn_view_left.clicked.connect(self.gl_window.set_view_left)
307 self.btn_view_top.clicked.connect(self.gl_window.set_view_top)
```

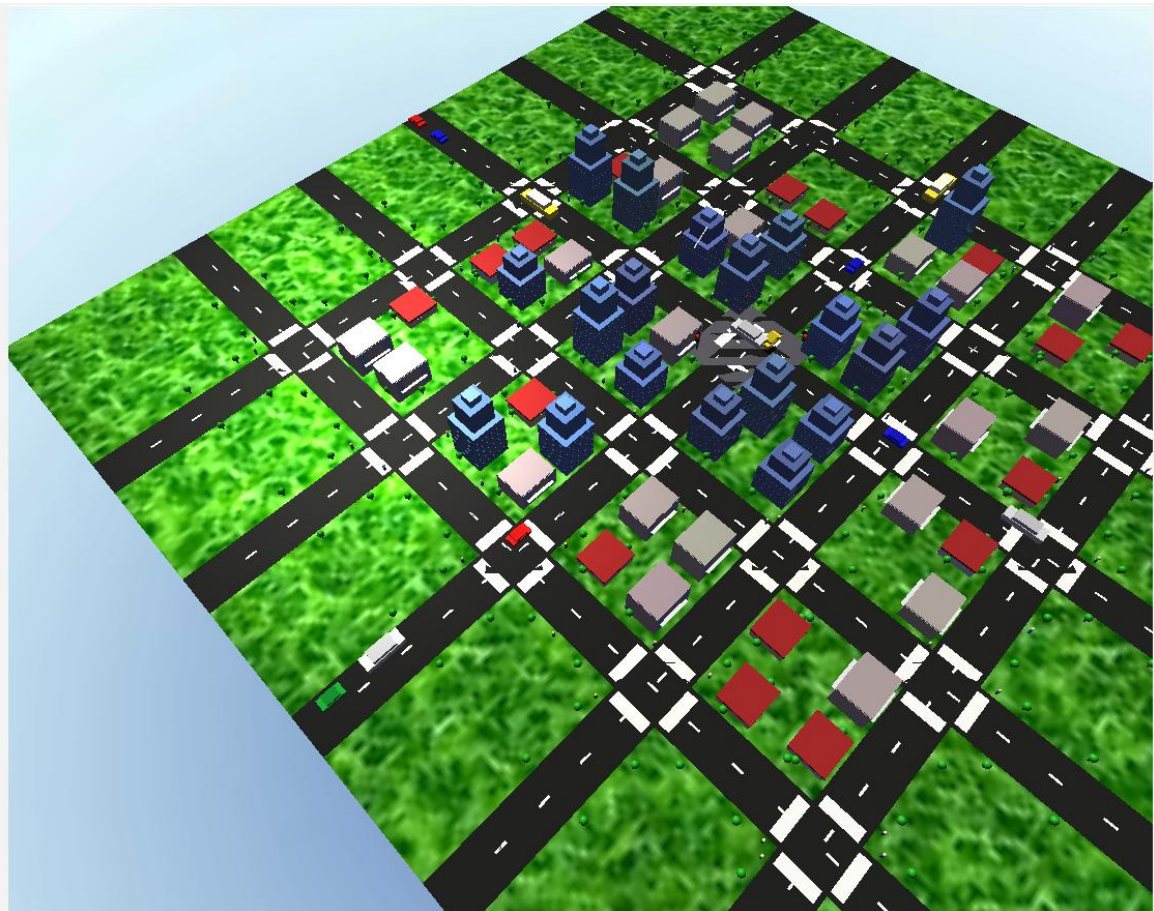
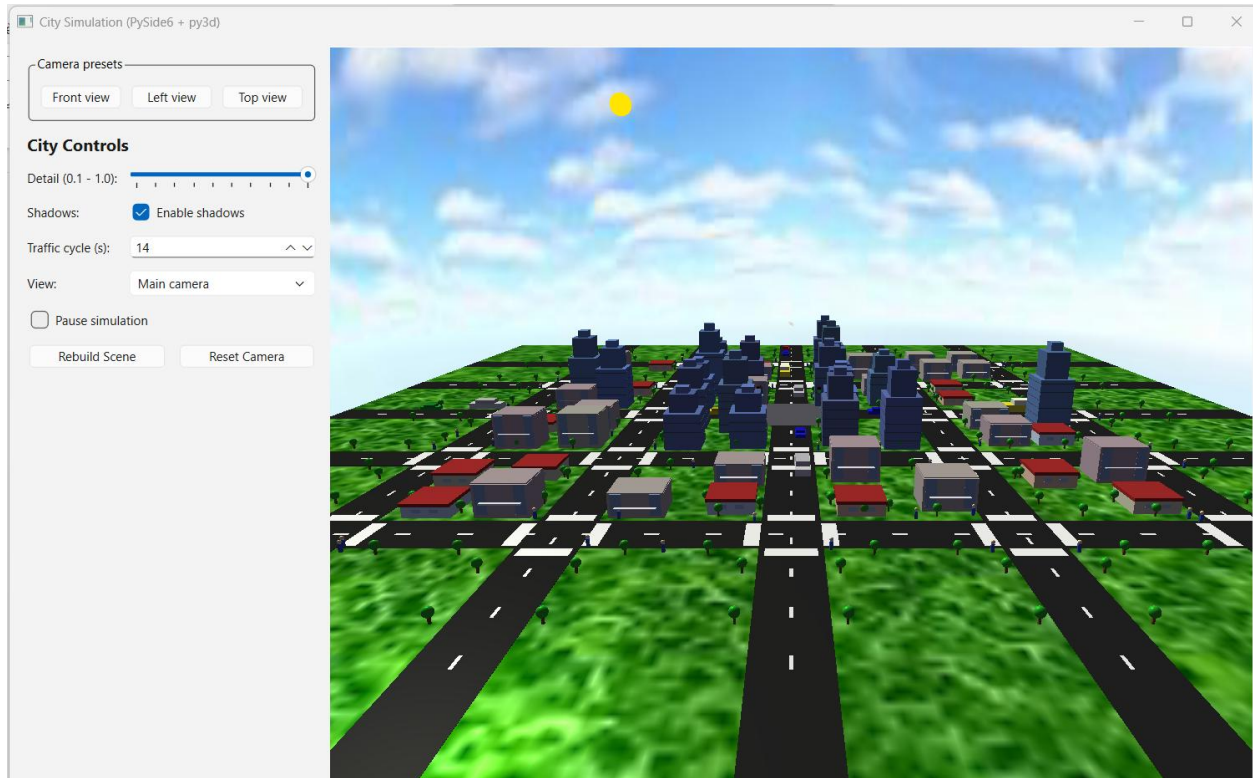
2.2. Slots in the GL window

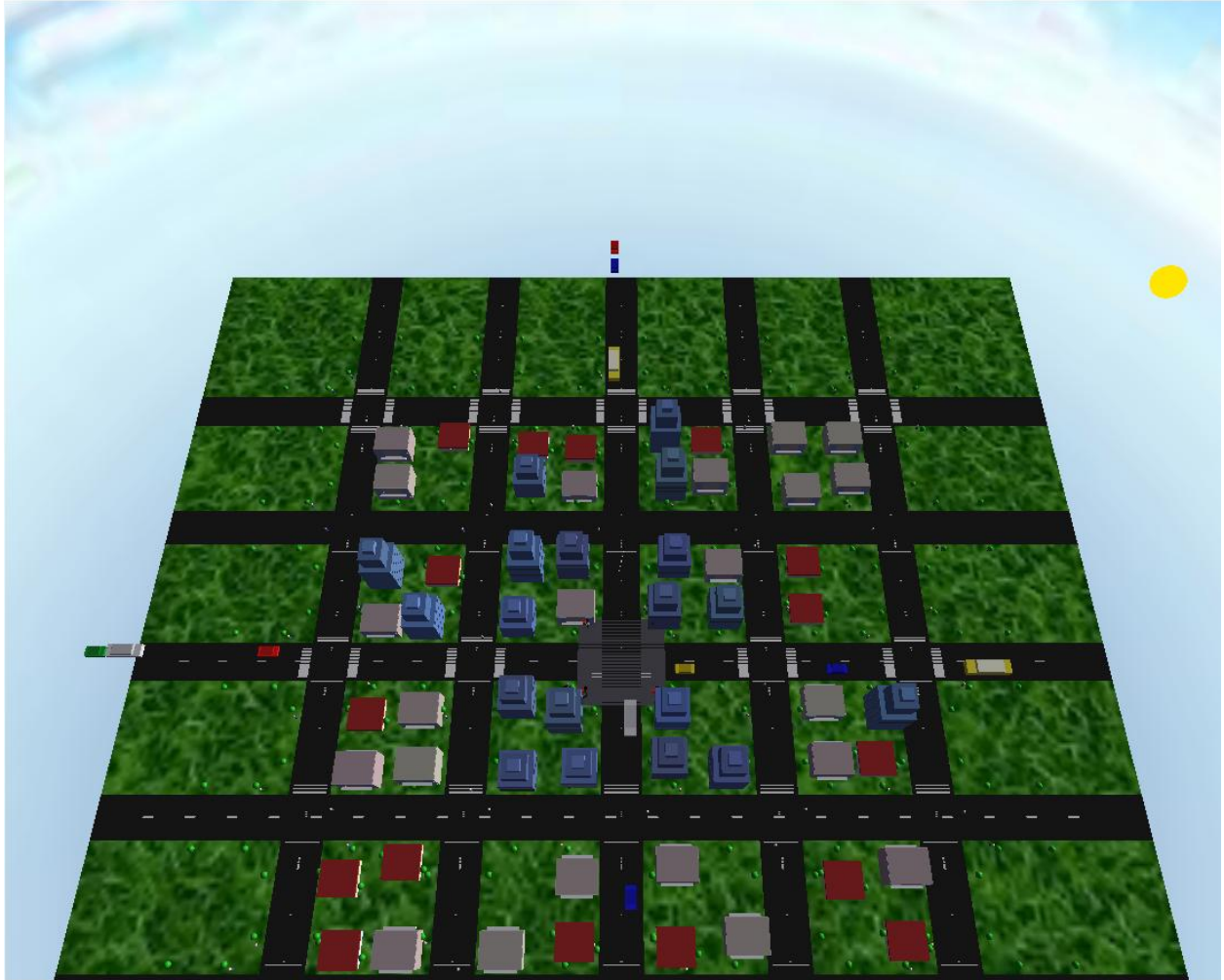
```
lab8_gui.py > CityGLWindow > set_view_left
55 class CityGLWindow(QOpenGLWindow):
239     def _on_tick(self):
252
253         self.engine.step(dt, input_state=None)
254         self.update()
255
256     def set_view_front(self):
257         if self.engine is None:
258             return
259         self.engine.rig.set_position([0, 12, 45])
260         self.engine.camera.look_at([0, 0, 0])
261         self.update()
262
263     def set_view_left(self):
264         if self.engine is None:
265             return
266         self.engine.rig.set_position([-45, 12, 0])
267         self.engine.camera.look_at([0, 0, 0])
268         self.update()
269
270     def set_view_top(self):
271         if self.engine is None:
272             return
273         self.engine.rig.set_position([0, 80, 0])
274         self.engine.camera.look_at([0, 0, 0])
275         self.update()
```

2.3. Connect signals and slots

```
lab8_gui.py X
lab8_gui.py > ControlPanel > __init__
281 class ControlPanel(QWidget):
282     def __init__(self, gl_window: CityGLWindow, parent=None):
293
294         self.btn_view_front = QPushButton("Front view")
295         self.btn_view_left = QPushButton("Left view")
296         self.btn_view_top = QPushButton("Top view")
297
298         h.addWidget(self.btn_view_front)
299         h.addWidget(self.btn_view_left)
300         h.addWidget(self.btn_view_top)
301
302         group.setLayout(h)
303         layout.addWidget(group)
304
305         self.btn_view_front.clicked.connect(self.gl_window.set_view_front)
306         self.btn_view_left.clicked.connect(self.gl_window.set_view_left)
307         self.btn_view_top.clicked.connect(self.gl_window.set_view_top)
308
309         title = QLabel("City Controls")
310         title.setStyleSheet("font-weight: bold; font-size: 16px;")
311         layout.addWidget(title)
312
313         form = QFormLayout()
314         form.setLabelAlignment(Qt.AlignLeft)
315         form.setFormAlignment(Qt.AlignTop)
316
317         # Detail slider: 0.1 -> 1.0
318         self.detail_slider = QSlider(Qt.Horizontal)
```

Result





Description:

Three camera presets buttons are added: **Front View**, **Left View**, and **Top View**.

Each button calls the OpenGL, camera rig position and camera look_at.

Task 3 – Camera movement buttons

3.1. Add movement buttons


```

318
319     # --- Camera Movement ---
320     self.btn_forward = QPushButton("Forward")
321     self.btn_backward = QPushButton("Backward")
322     self.btn_left = QPushButton("Left")
323     self.btn_right = QPushButton("Right")

```

3.2. Implement movement in the GLwindow or cityengine

```

275         self.update()
276
277     def move_camera(self, dx=0.0, dy=0.0, dz=0.0):
278         if self.engine is None:
279             return
280
281         pos = list(self.engine.rig.global_position)
282         pos[0] += dx
283         pos[1] += dy
284         pos[2] += dz
285         self.engine.rig.set_position(pos)
286

```

```

324
325     self.btn_forward.clicked.connect(
326         lambda: self.gl_window.move_camera(0, 0, -5)
327     )
328     self.btn_backward.clicked.connect(
329         lambda: self.gl_window.move_camera(0, 0, 5)
330     )
331     self.btn_left.clicked.connect(
332         lambda: self.gl_window.move_camera(-5, 0, 0)
333     )
334     self.btn_right.clicked.connect(
335         lambda: self.gl_window.move_camera(5, 0, 0)
336     )
337

```



3.3. Short explanation

I implemented the `move_camera` function inside the OpenGL window, where the camera rig's global position is updated by adding a fixed offset along the x, y, or z axes. The movement buttons in the GUI are connected using lambda expressions that call `move_camera` with predefined step values. In my city coordinate system, the **z-axis** represents forward and backward movement (negative z = forward, positive z = backward). The **x-axis** is used for left and right movement (negative x = left, positive x = right). The optional vertical movement uses the **y-axis**, where positive values move the camera upward.

Task 4 – Extra feature

Automatic camera tour

```
241  def toggle_tour(self, checked):
242      self.auto_tour = checked
243      if checked:
244          self._tour_start_time = self.engine.time
245          print("Tour started")
246      else:
247          self.reset_camera()
248          print("Tour stopped")
249
```

```
252  def on_tick(self):
253      if self.engine is None:
254          return
255
256      now = time.perf_counter()
257      dt = now - self.last_time if self.last_time is not None else 0.0
258      self.last_time = now
259
260      self.engine.step(dt, input_state=None)
261
262      if self.auto_tour:
263          t = self.engine.time
264          radius = 60.0
265          speed = 0.4
266
267          x = radius * math.cos(0.1 * t)
268          z = radius * math.sin(0.1 * t)
269
270          self.engine.rig.set_position([x, 20, z])
271          self.engine.camera.look_at([0, 0, 0])
272      self.update()
```

Description: The automatic camera follows a **circular path** around the city. The camera's position is updated over time using sine and cosine functions, which create a smooth orbit at a fixed radius and constant height above the ground. While moving along this circular trajectory, the camera always looks toward the center of the scene, allowing the entire city to remain in view throughout the motion. This results in a

continuous and stable panoramic tour of the environment. A screenshot captured during the automatic camera tour is shown below.

